Lab 7 — Capstone: Telecom Site Monitor

Time: ~18 minutes in the Telecom & Data-Centre Edition, where it is the default capstone in Module 8 (~35 minutes on your own). Teams who prefer a screen can do Lab 6 instead. **You'll learn:** how the same sense → send → decide → act loop runs a remote mobile-network site: its batteries, its generator, its fuel and its door.

> **Telecom & Data-Centre Edition**
>
> **The story:** A tower site on a hilltop, hours from the nearest engineer. When the grid fails, the backup batteries take over; when they run low, the generator must start. Someone has to notice if fuel disappears, or if the shelter door opens at night.
>
> **Your team:** Teams of three. Rotate the roles from the last lab: **builder** (keyboard), **analyst** (reads the guide, checks the output), **decision owner** (answers the manager task).
>
> **Manager task:** The site raises three kinds of alarm. Who should receive each one, and what should they do first?
>
> **Business-case canvas:** After the lab, fill in boxes 6 (costs) and 7 (payback), then prepare your 2-minute pitch.

What you're building

One ESP32 that watches a tower site and acts on its own:

Part in the simulator	Stands for	What the code does with it
Left dial (potentiometer)	The voltage of the 48 V backup battery bank	Starts the generator below 47 V and stops it above 52 V
Right dial (potentiometer)	The level in the generator's fuel tank	Raises a *possible fuel theft* alarm if fuel drops while the generator is off
Slide switch	The shelter door contact	Raises an alarm when the door opens
DHT22 sensor	The shelter's temperature	Reports it
Relay + orange LED	The generator's start switch	Turns on when the generator runs
Buzzer	The site's local alarm	Sounds during an alarm

Every reading, and the site's status, goes to the operations centre over MQTT, exactly as in Lab 3. Watch them arrive in the HiveMQ web client.

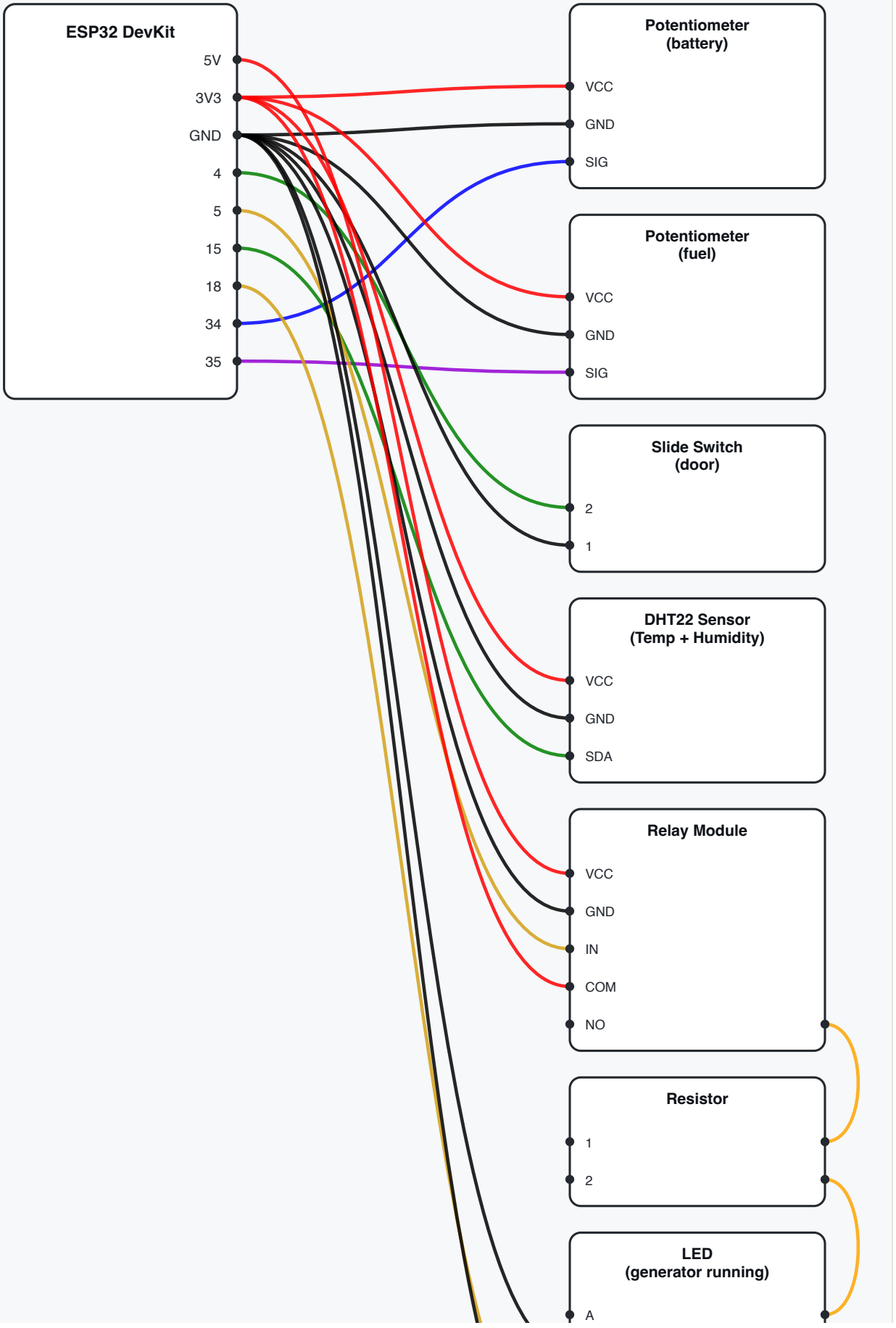
Three ideas from the course meet here:

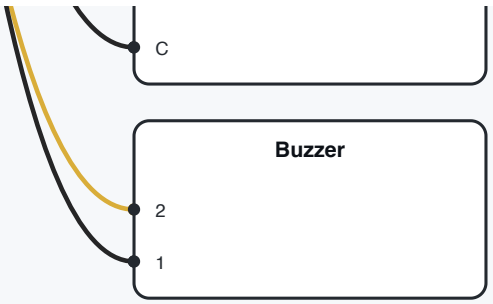
- **Edge decisions** (Module 6): the generator starts at the site, even if the link to the operations centre is down.
- **Hysteresis** (Module 6): start below 47 V, stop only above 52 V, so the generator doesn't start and stop every few seconds.
- **Comparing two readings** (Module 6): fuel *should* only fall while the generator runs. Fuel that falls while it is off is suspicious.

Wiring diagram

Lab 7 — Telecom Site Monitor Wiring

Generated from this lab's real diagram.json — colors match the wires you'll paste into Wokwi





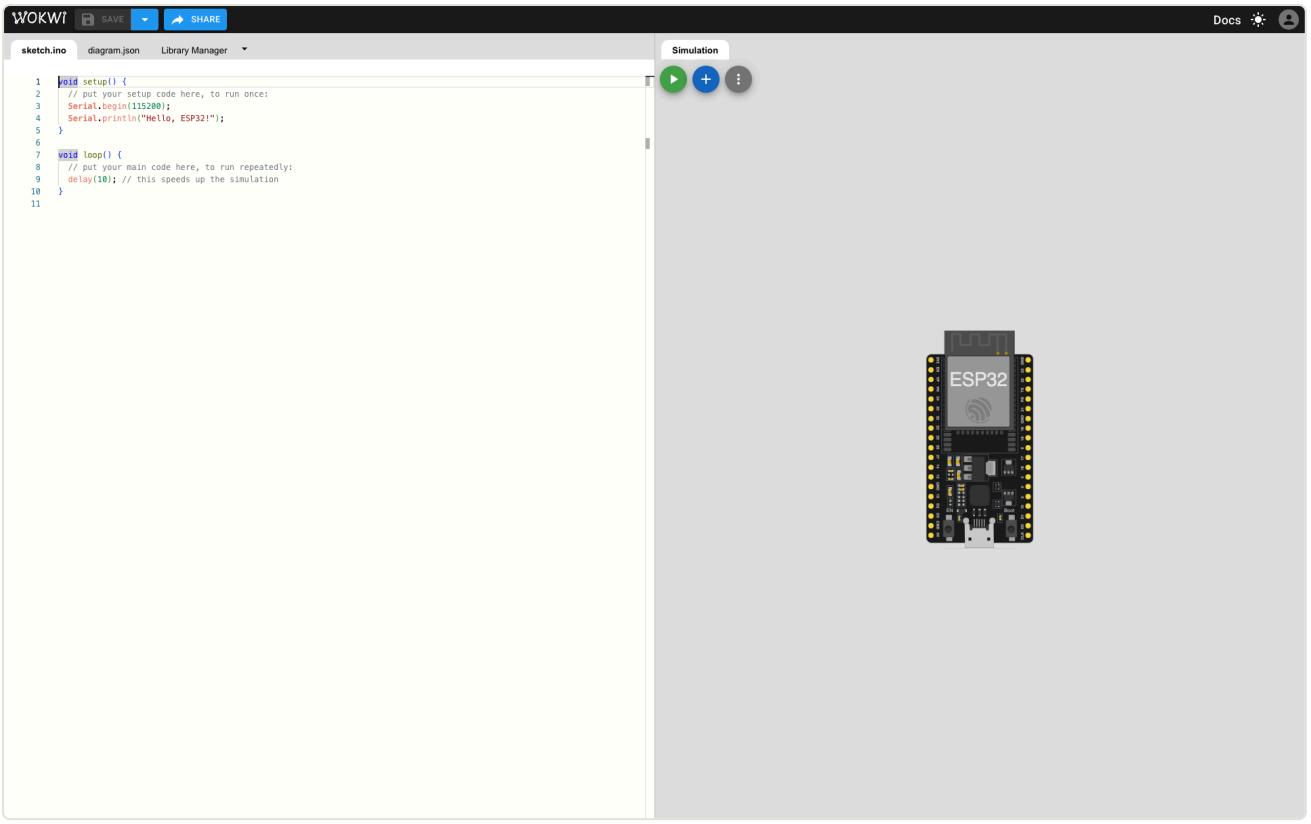
ESP32 Pin	Wire Color	Connects To
`34`	blue	Left dial, battery (SIG)
`35`	violet	Right dial, fuel (SIG)
`4`	green	Slide switch, door (middle pin)
`15`	green	DHT22 (data pin)
`5`	yellow	Relay Module (IN)
`18`	yellow	Buzzer
`3V3`	red	Both dials (VCC), DHT22 (VCC), Relay Module (COM)
`5V`	red	Relay Module (VCC)
`GND`	black	Both dials, slide switch, DHT22, Relay Module, LED, Buzzer

The dials use pins 34 and 35 because they keep working while Wi-Fi is on (Module 3: use the ADC1 pins, GPIO 32 to 39).

Step-by-step

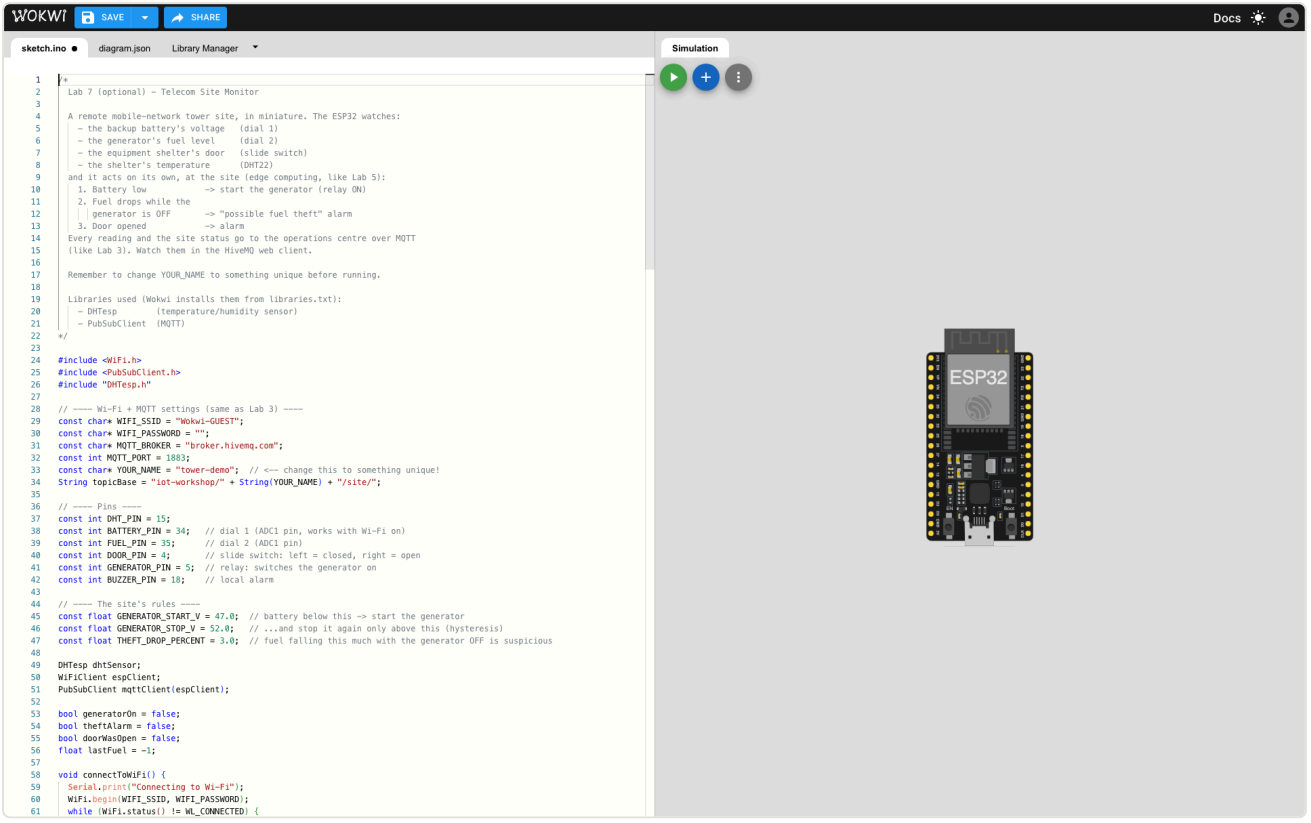
1. Open a new ESP32 project

Go to wokwi.com/projects/new/esp32.



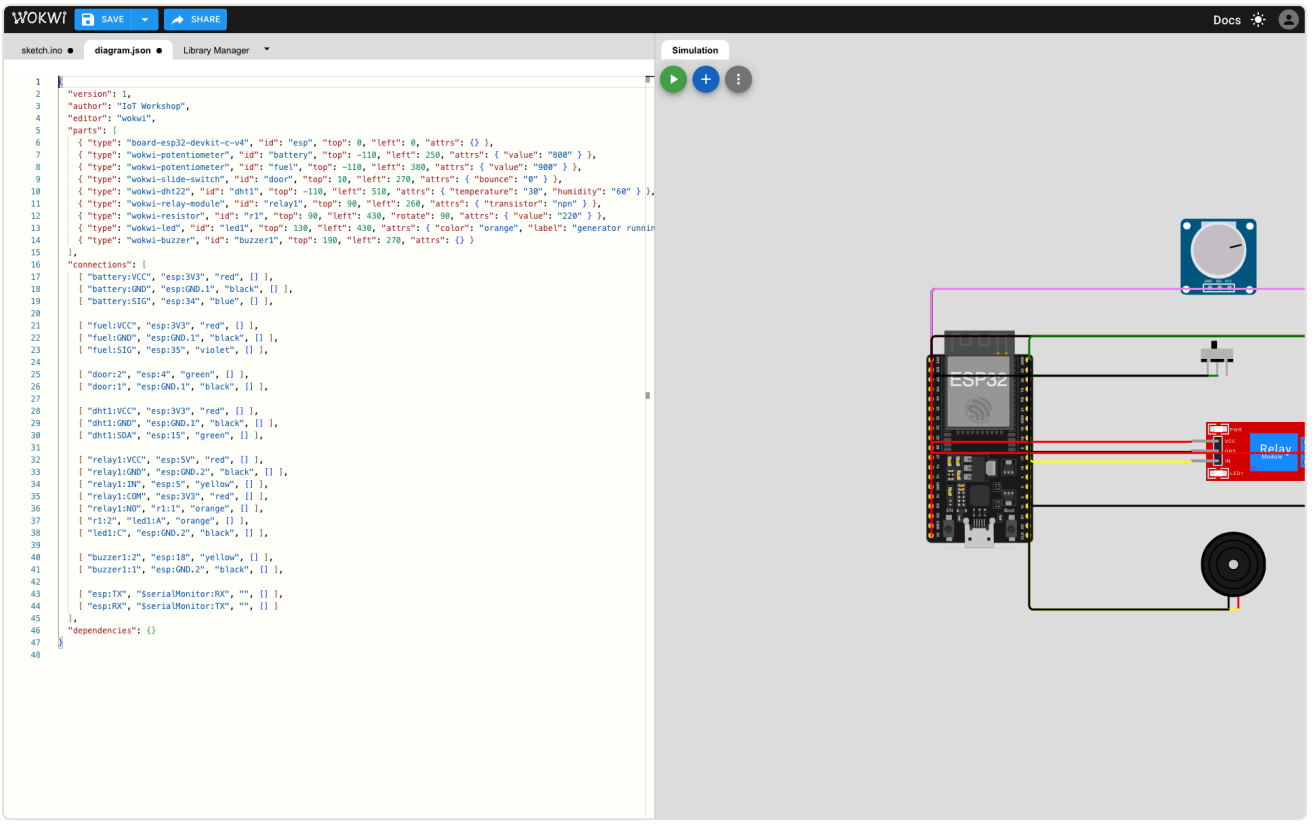
2. Paste the code, and change one line

Copy `sketch.ino` into Wokwi's **sketch.ino** tab. Before pressing Play, change `YOUR_NAME` near the top to something unique, exactly like in Lab 3 (the pictures use `tower-demo`).



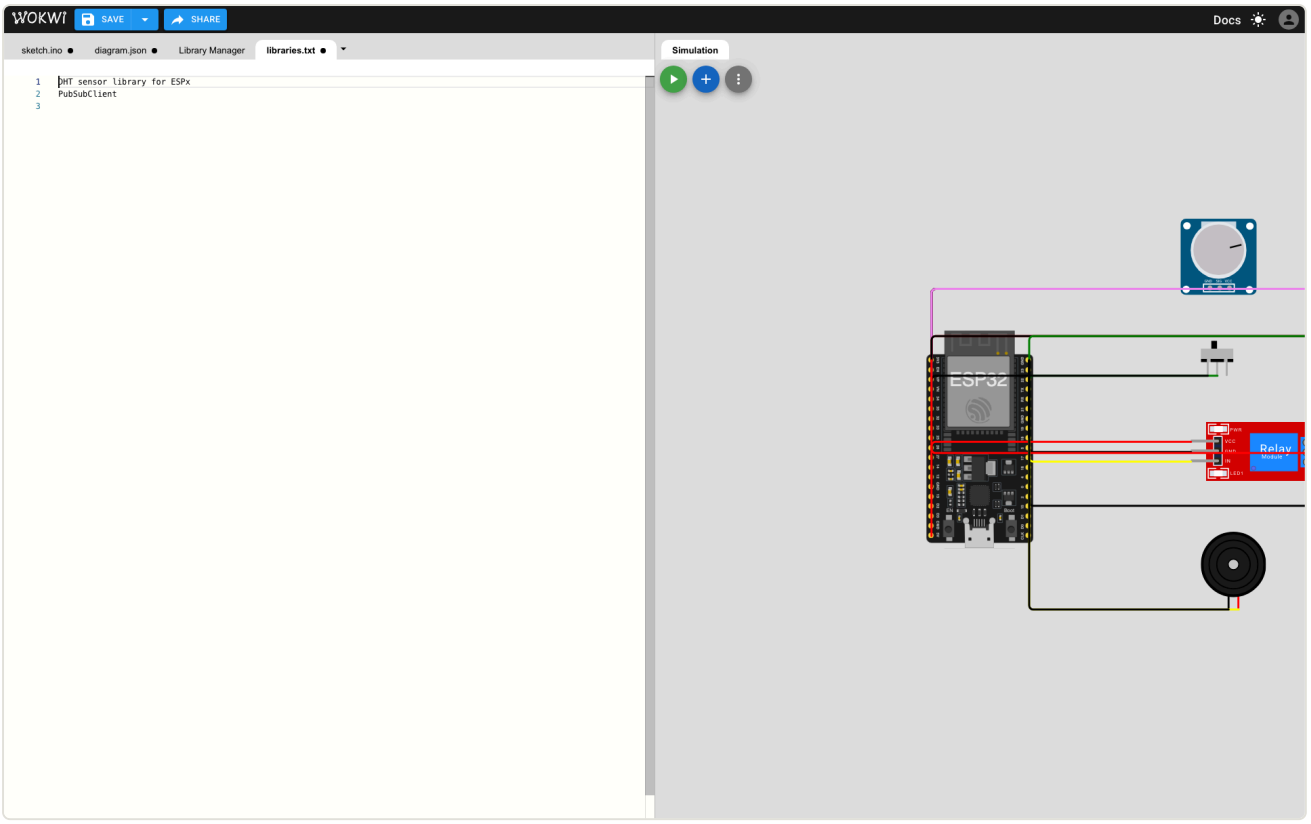
3. Paste the circuit

Copy `diagram.json` into Wokwi's **diagram.json** tab. You should see the two dials, the slide switch, the DHT22, the relay with its orange "generator running" LED, and the buzzer.



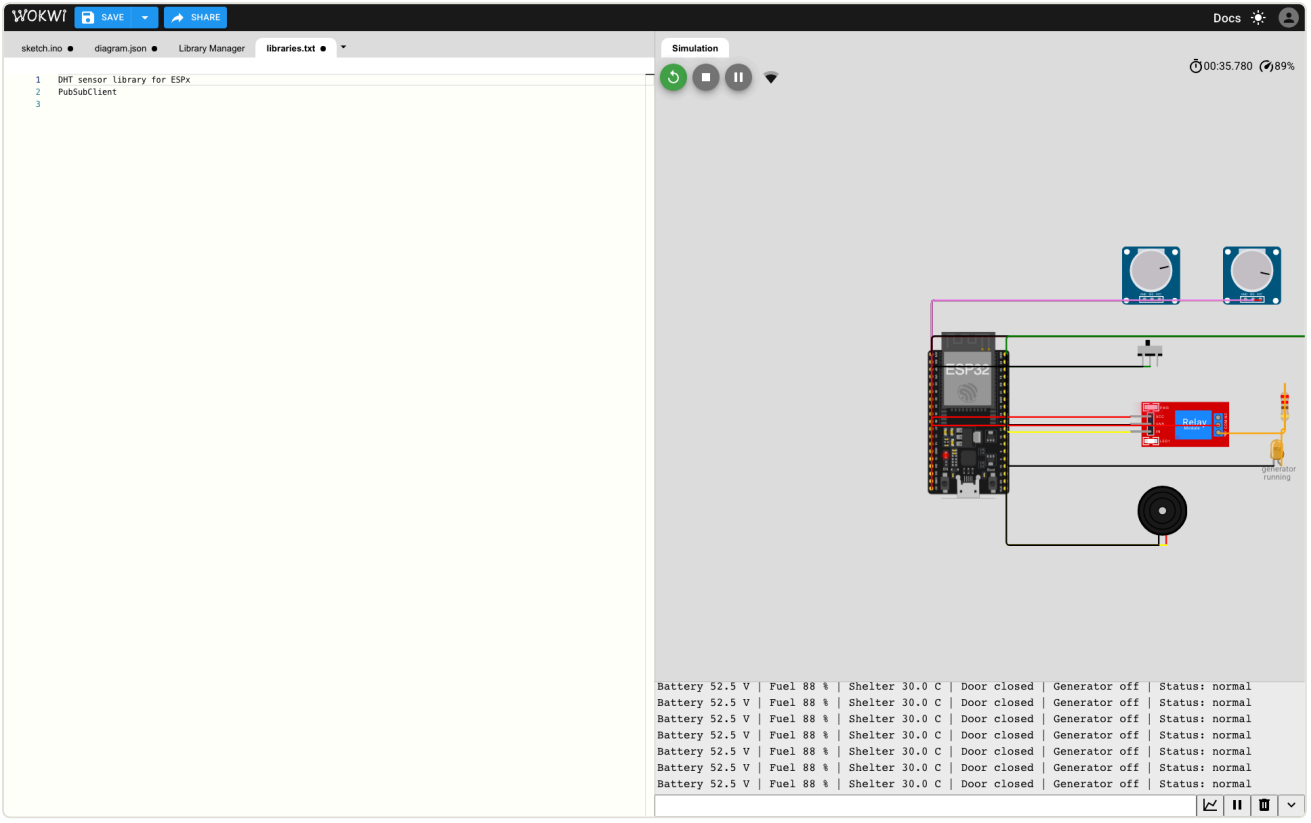
4. Add the library list

Two libraries, the same as Lab 3: click the small ▾ next to the **Library Manager** tab, choose **New file...**, name it `libraries.txt`, and paste in the contents of `libraries.txt`.



5. Press Play

The first build can take a minute. If you see **“Build Servers Busy”**, close the message and press Play again.



What you should see

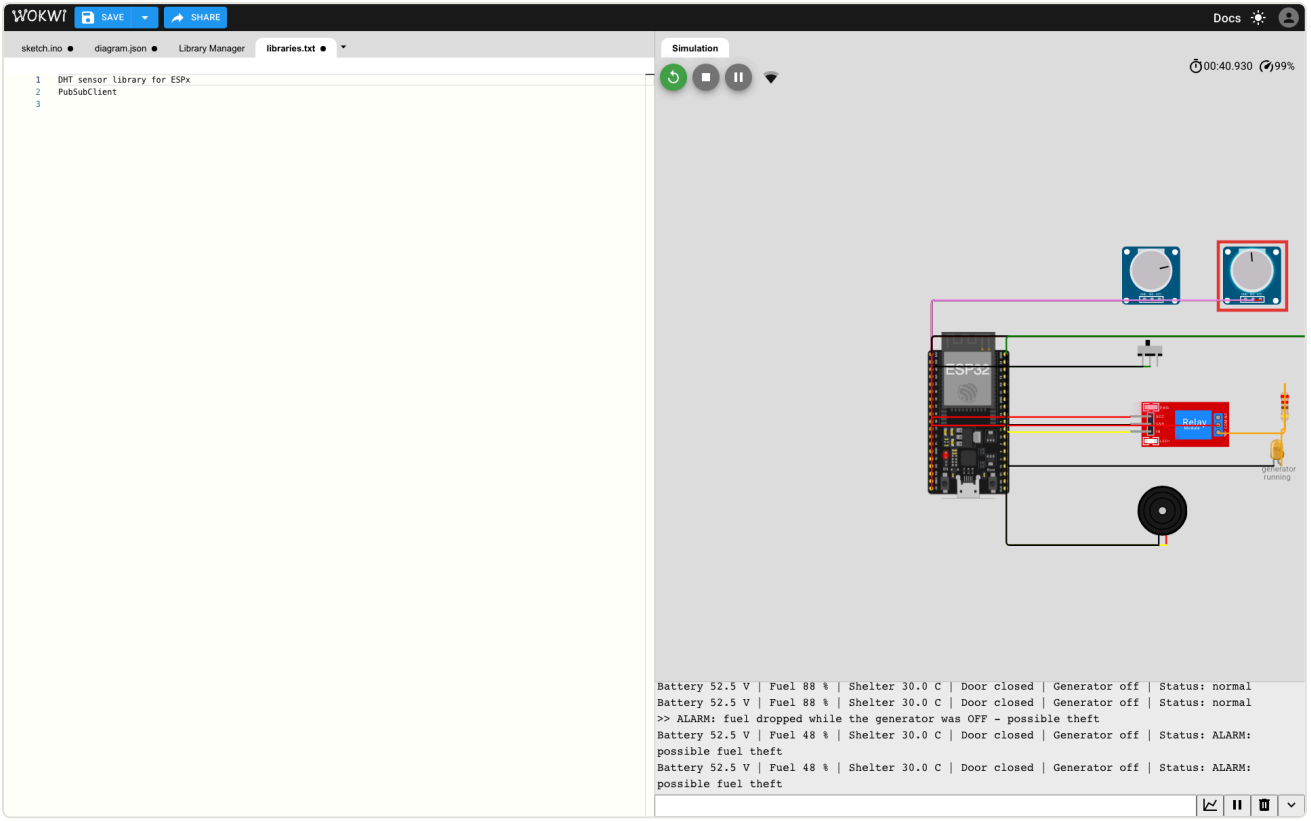
Serial Monitor:

```
Connecting to Wi-Fi..... connected!
MQTT connected!
Telecom Site Monitor started.
Publishing to: iot-workshop/tower-demo/site/#
Battery 52.5 V | Fuel 88 % | Shelter 30.0 C | Door closed | Generator off | Status:
normal
```

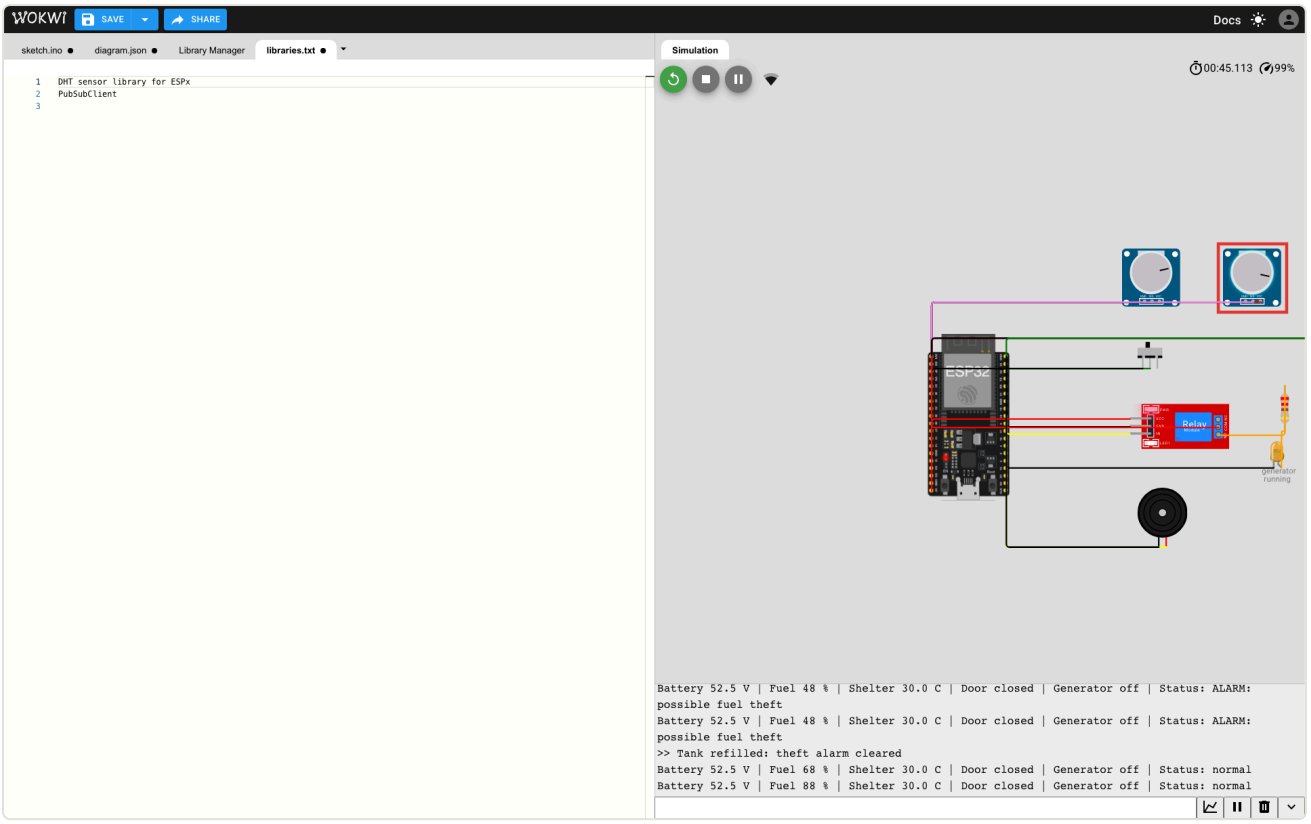
Try it

Do these in order. Click a dial first, then use the **Page Down** and **Page Up** keys (or drag the dial) to turn it.

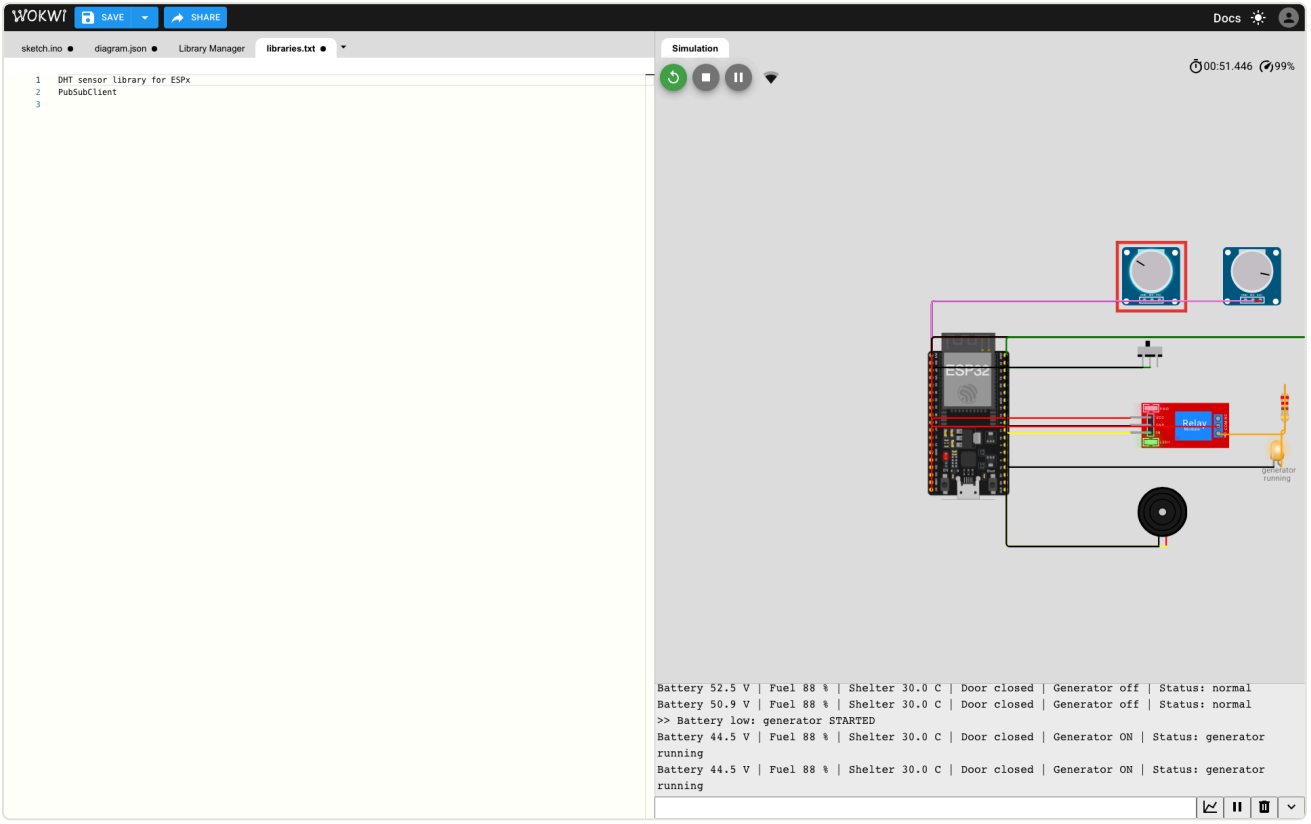
1. **Steal some fuel.** Turn the fuel dial (the right one) down while the generator is off. The site raises an alarm and the buzzer sounds.



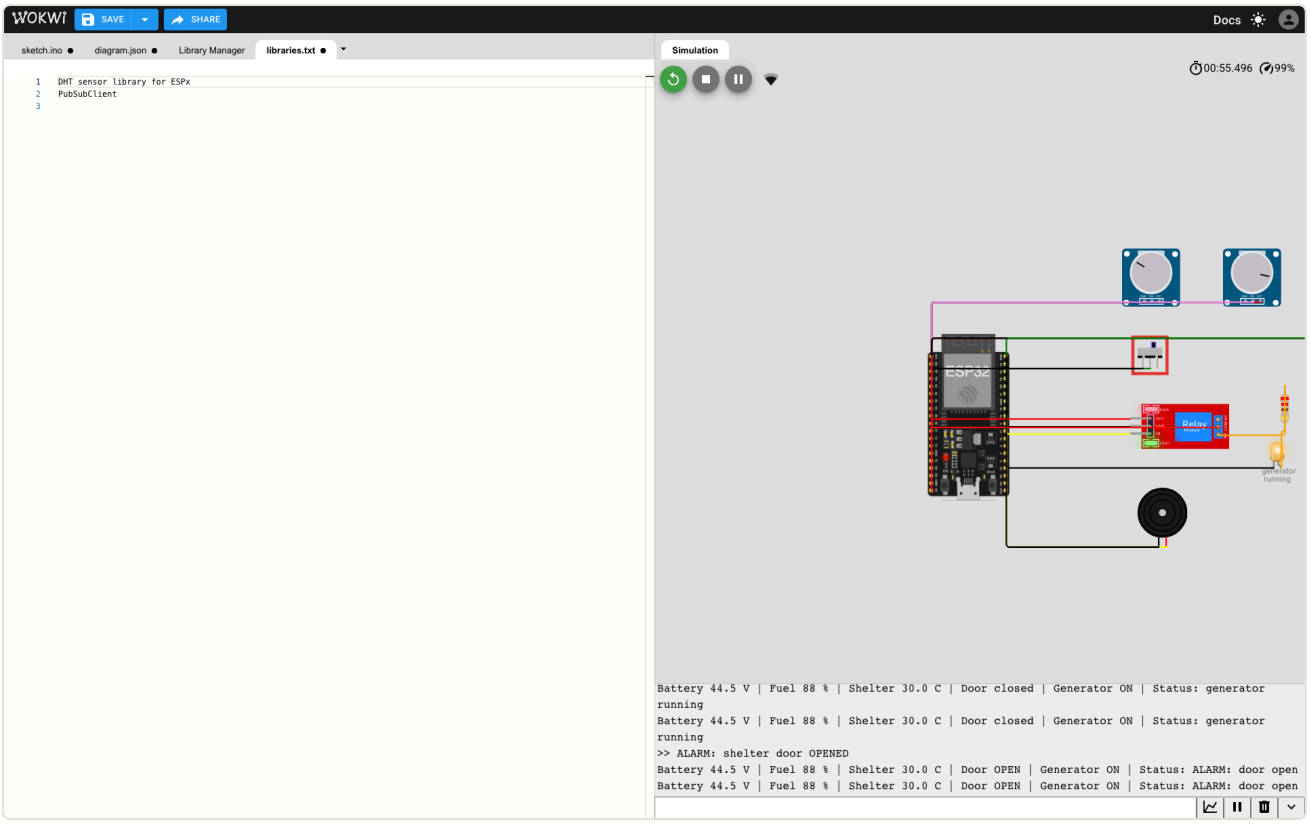
2. **Refill the tank.** Turn the fuel dial back up. The alarm clears.



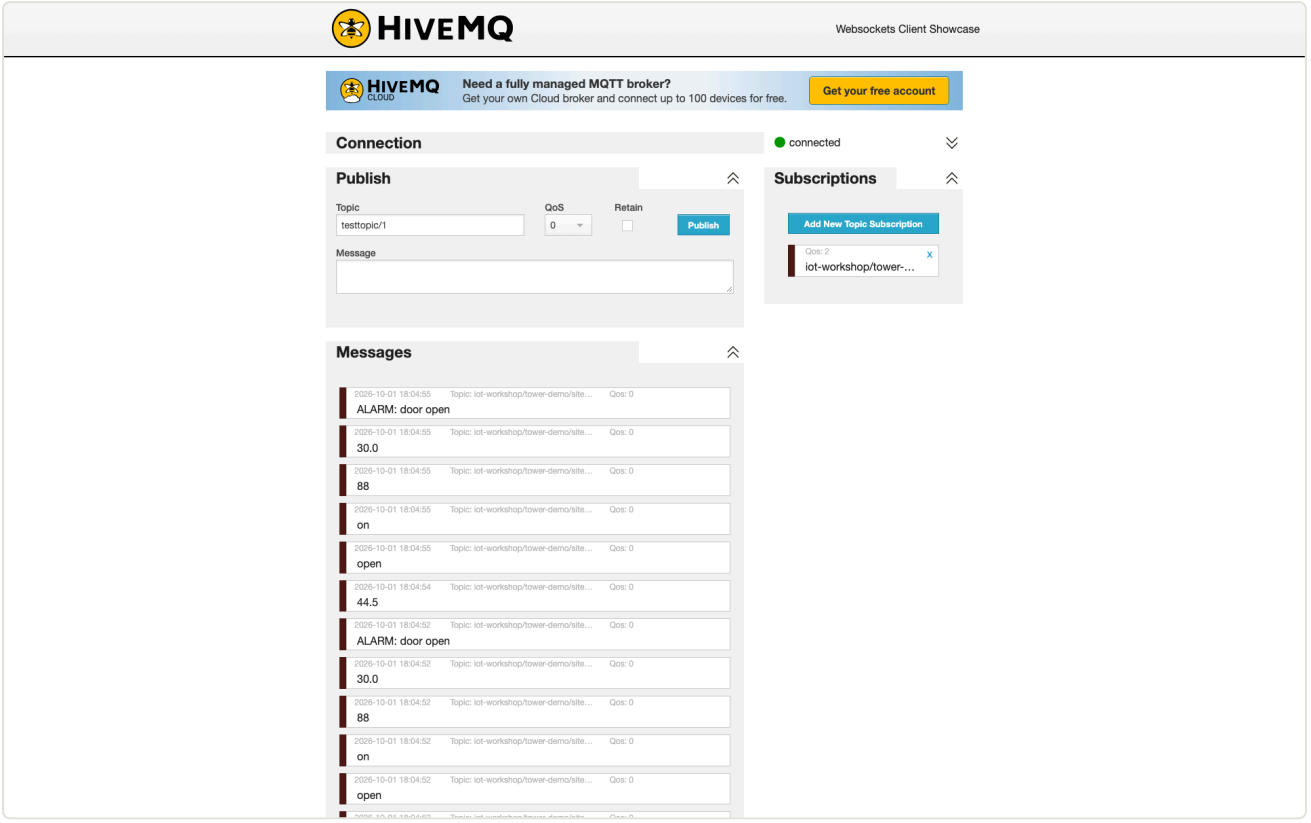
3. **Drain the battery.** Turn the battery dial (the left one) down below 47 V. The generator starts: the relay switches and the orange LED lights. Fuel used *now* is normal, so no theft alarm.



4. **Open the door.** Click the slide switch. The site reports the open door and the buzzer sounds.



5. **Watch it from the operations centre.** In the HiveMQ web client, connect and subscribe to `iot-workshop/YOUR_NAME/site/#`. Every reading and every status change arrives there.



Extension (fast teams)

- **Heat alarm.** Make a shelter temperature above 40 °C a third alarm reason. Find the line `String status = "normal";` and add your own rule below the door rule.
- **Discuss:** the fuel-theft rule uses a drop of 3 percentage points. What would go wrong if you made it 0.5? What about 30?

Troubleshooting

- **The dials don’t move with the keys.** Click the dial once first, so it has the keyboard focus.
- **A theft alarm when you didn’t expect one.** The generator was off and the fuel dial moved down by more than 3 points between two readings. That’s the rule working; turn the dial back up to clear it.
- **MQTT won’t connect.** Check that you changed `YOUR_NAME`, and wait a few seconds: the code retries on its own.